

# Módulo 3 - Comunicación SPI para el display TFT

Proyecto de identificación RLS/ARX en STM32F405RGT6

Archivos documentados: spi\_driver.h, spi\_driver.c, spi\_if.h y spi\_if.c.



**Función del módulo.** Implementa el canal serie síncrono que permite enviar comandos y datos al controlador gráfico ILI9341.

**Periférico usado.** SPI1 del STM32F405, con SCK = PA5 y MOSI = PA7. La línea MISO = PA6 no se utiliza en esta aplicación.

**Relación con otros módulos.** El SPI mueve los bytes; el bloque GPIO aporta CS, DC y RST; la librería ILI9341\_driver define qué secuencia de bytes se transmite.

**Patrón de diseño.** El archivo spi\_if envuelve el driver con una interfaz basada en punteros a función para desacoplar la aplicación del detalle de hardware.

## 1. Papel del módulo dentro del firmware

Este módulo existe para resolver una necesidad muy concreta del proyecto: enviar información desde el microcontrolador hacia la pantalla TFT. El algoritmo RLS calcula datos y la librería del ILI9341 decide qué dibujar, pero sin SPI no existiría un canal físico para entregar esos bytes al display.

Didácticamente, este bloque es valioso porque muestra una situación frecuente en sistemas embebidos: el periférico de comunicación no trabaja solo. Aquí el SPI transporta los bytes, mientras que varias señales auxiliares de GPIO determinan si cada byte debe interpretarse como comando o como dato y cuándo el display está seleccionado.

## 2. Relación entre archivos

| Archivo      | Responsabilidad                                                                                              |
|--------------|--------------------------------------------------------------------------------------------------------------|
| spi_driver.h | Declara la API mínima del driver: inicialización de pines, inicialización del periférico y envío de un byte. |
| spi_driver.c | Programa los registros de GPIOA y SPI1 para poner en marcha la interfaz serie síncrona.                      |
| spi_if.h     | Define una estructura de interfaz con punteros a función para encapsular el módulo SPI.                      |
| spi_if.c     | Construye la instancia global constante SPI1_STM32 y enlaza la capa de aplicación con el driver real.        |

Fragmento 1. API publicada por spi\_driver.h

```
void SPI1_gpio_init(void);
void SPI1_init(void);
void SPI1_send_byte(uint8_t data);
```

### 3. Contexto hardware del SPI1

El driver emplea el periférico SPI1, que en este microcontrolador pertenece al dominio APB2. Las líneas físicas elegidas son PA5 como reloj serie y PA7 como salida de datos del maestro. El comentario del propio archivo menciona PA6 como MIS0, pero aclara que no se usa para este caso.

| Señal  | Pin STM32 | Dirección   | Uso en el proyecto                                                                       |
|--------|-----------|-------------|------------------------------------------------------------------------------------------|
| SCK    | PA5       | Salida      | Genera el reloj de la transferencia SPI hacia el ILI9341.                                |
| MOSI   | PA7       | Salida      | Entrega los bytes de comandos y datos que dibujan la interfaz gráfica.                   |
| MISO   | PA6       | Entrada     | No participa en el firmware actual porque la pantalla se usa esencialmente en escritura. |
| NSS/CS | PB6       | Salida GPIO | No lo maneja el periférico SPI; se controla manualmente desde el módulo GPIO.            |

#### 3.1 ¿Por qué el chip select no lo maneja el hardware SPI?

La pantalla usa una línea CS externa que el proyecto gobierna con GPIO. Esa decisión es común cuando un periférico gráfico necesita además otras líneas de control como DC y RST, o cuando el diseñador quiere tener control completo del momento exacto en que el esclavo queda activo.

### 4. Inicialización de pines: SPI1\_gpio\_init()

La primera rutina no configura todavía el periférico SPI; solo prepara los pines del puerto A para que funcionen como líneas alternativas del bus. El flujo es clásico: habilitar el reloj del puerto, cambiar el modo de los pines a Alternate Function, asignar la función alternativa correcta y ajustar características eléctricas básicas.

#### Fragmento 2. Configuración de GPIOA para SPI1

```
void SPI1_gpio_init(void)
{
    RCC->AHB1ENR |= GPIOAENb;

    GPIOA->MODER &= ~0x0000CC00U;
    GPIOA->MODER |= 0x00008800U;

    GPIOA->AFR[0] &= ~0xF0F00000U;
    GPIOA->AFR[0] |= 0x50500000U;

    GPIOA->OSPEEDR |= 0x0000CC00U;
    GPIOA->OTYPER &= ~0x000000A0U;
    GPIOA->PUPDR &= ~0x0000CC00U;
}
```

| Registro      | Valor o máscara | Interpretación didáctica                                                  |
|---------------|-----------------|---------------------------------------------------------------------------|
| RCC->AHB1ENR  | GPIOAENb        | Habilita el reloj del puerto A, condición necesaria antes de tocar GPIOA. |
| GPIOA->MODER  | 0x00008800      | Pone PA5 y PA7 en modo Alternate Function (10b).                          |
| GPIOA->AFR[0] | 0x50500000      | Asigna AF5, que es la función correspondiente a SPI1 en esos pines.       |

| Registro       | Valor o máscara  | Interpretación didáctica                                                                |
|----------------|------------------|-----------------------------------------------------------------------------------------|
| GPIOA->OSPEEDR | 0x0000CC00       | Selecciona alta velocidad para mejorar integridad de señal en un bus serie rápido.      |
| GPIOA->OTYPER  | clear 0xA0       | Deja los pines en push-pull, apropiado para señales digitales activas de reloj y datos. |
| GPIOA->PUPDR   | clear 0x0000CC00 | Desactiva resistencias internas para no alterar innecesariamente la línea.              |

#### 4.1 Lectura rápida de la máscara de MODER

La máscara 0x0000CC00 cubre dos pares de bits: [11:10] para PA5 y [15:14] para PA7. Primero se limpian y luego se cargan con 10b, que representa el modo Alternate Function. Este tipo de lectura de máscaras ayuda al alumno a no perderse cuando el código se escribe sin usar macros individuales por pin.

## 5. Inicialización del periférico: SPI1\_init()

Después de preparar los pines, la segunda rutina habilita el bloque SPI1 y define sus parámetros básicos de operación. El código usa principalmente el registro CR1. Cada bit importante de este registro cambia un aspecto concreto del protocolo: el modo, el orden de bits, el rol maestro/esclavo, el formato de palabra y la estrategia para manejar la señal de selección de esclavo.

#### Fragmento 3. Secuencia de configuración del registro CR1

```
void SPI1_init(void)
{
    RCC->APB2ENR |= SPI1ENB;

    SPI1->CR1 &= ~0x0038U;
    SPI1->CR1 |= 0x0010U;

    SPI1->CR1 &= ~0x0003U;
    SPI1->CR1 &= ~0x0400U;
    SPI1->CR1 &= ~0x0080U;
    SPI1->CR1 |= 0x0004U;
    SPI1->CR1 &= ~0x0800U;
    SPI1->CR1 |= 0x0300U;
    SPI1->CR1 |= 0x0040U;
}
```

| Campo                | Acción en el código      | Resultado                                                                      |
|----------------------|--------------------------|--------------------------------------------------------------------------------|
| Reloj del periférico | RCC->APB2ENR  = SPI1ENB  | Activa el bloque SPI1 en el bus APB2.                                          |
| BR[2:0]              | clear 0x0038; set 0x0010 | Programa el prescaler del reloj serie. Según el valor escrito, BR = 010.       |
| CPOL, CPHA           | clear 0x0003             | Selecciona modo SPI 0: reloj inactivo en bajo y muestreo en flanco ascendente. |
| RXONLY               | clear 0x0400             | No deja el periférico en recepción solamente.                                  |
| LSBFIRST             | clear 0x0080             | Transmisión MSB first.                                                         |
| MSTR                 | set 0x0004               | El STM32 opera como maestro del bus.                                           |
| DFF                  | clear 0x0800             | Tamaño de palabra de 8 bits.                                                   |

| Campo    | Acción en el código | Resultado                                                           |
|----------|---------------------|---------------------------------------------------------------------|
| SSM, SSI | set 0x0300          | Selección del esclavo por software; evita depender de NSS hardware. |
| SPE      | set 0x0040          | Habilita finalmente el periférico SPI.                              |

### 5.1 Nota importante sobre la velocidad SPI

Aquí conviene hacer una lectura crítica del código. El comentario dentro del archivo dice que el ajuste equivale a  $f_{PCLK}/16 = 5.25 \text{ MHz}$ , pero el valor realmente escrito en los bits BR es 010, que en la familia STM32F4 corresponde a  $f_{PCLK}/8$ . Si  $APB2 = 84 \text{ MHz}$ , el reloj SPI efectivo sería  $10.5 \text{ MHz}$ . Esta discrepancia es excelente material para clase porque enseña al alumno que el valor real lo define el registro, no el comentario.

Observación didáctica. Si se quisiera asegurar un reloj menor a 10 MHz de forma conservadora para el ILI9341, sería más coherente programar  $BR = 011$  (división entre 16) o verificar explícitamente el límite admitido por el módulo TFT utilizado en laboratorio.

### 5.2 Un detalle de implementación: bus de solo escritura

El proyecto usa el display como receptor de comandos y datos, de modo que la lógica de aplicación no lee respuestas del ILI9341. En el código, el periférico queda configurado en una forma funcional para transmitir, pero mantiene activa la naturaleza full-duplex del SPI. En proyectos más refinados, algunos diseñadores prefieren configurar un modo estrictamente de transmisión o limpiar la ruta de recepción para evitar banderas residuales de recepción no atendida.

## 6. Rutina de envío: SPI1\_send\_byte()

La tercera función del driver encapsula una transacción mínima de un byte. Primero espera a que el transmisor esté listo, luego escribe el dato directamente en el registro de datos con acceso de 8 bits y finalmente espera a que la cola de transmisión quede vacía y a que el periférico deje de estar ocupado.

#### Fragmento 4. Envío bloqueante de un byte por SPI

```
void SPI1_send_byte(uint8_t data)
{
    while (!(SPI1->SR & SPI_SR_TXE));
    *((volatile uint8_t *)&SPI1->DR) = data;
    while (!(SPI1->SR & SPI_SR_TXE));
    while (SPI1->SR & SPI_SR_BSY);
}
```

| Paso                              | Qué verifica     | Sentido práctico                                                            |
|-----------------------------------|------------------|-----------------------------------------------------------------------------|
| Espera de TXE antes de escribir   | $SR.TXE = 1$     | Garantiza que el buffer de transmisión acepta un nuevo byte.                |
| Escritura a DR                    | Acceso de 8 bits | Fuerza a que el dato se trate como byte y no como media palabra.            |
| Espera de TXE después de escribir | $SR.TXE = 1$     | Confirma que el registro de datos ya quedó libre de nuevo.                  |
| Espera de BSY en cero             | $SR.BSY = 0$     | Asegura que la transferencia física terminó en la línea antes de continuar. |

### 6.1 ¿Por qué escribir DR como puntero a uint8\_t?

Esta técnica es muy común en STM32 cuando se desea una transferencia explícita de 8 bits. El registro de datos admite distintos anchos de acceso, y castear su dirección a `volatile uint8_t *` evita ambigüedades sobre el tamaño de la escritura.

## 7. Capa de interfaz: spi\_if

La capa `spi_if` convierte el driver en un objeto de interfaz sencillo. En lugar de que la aplicación llame directamente a `SPI1_init()` o `SPI1_send_byte()`, usa una estructura constante con nombres más uniformes: `gpio_init`, `init` y `send_byte`.

**Fragmento 5. Definición de la interfaz en spi\_if.h**

```
typedef void (*spi_void_fn_t)(void);
typedef void (*spi_send_byte_fn_t)(uint8_t data);

typedef struct {
    spi_void_fn_t    gpio_init;
    spi_void_fn_t    init;
    spi_send_byte_fn_t send_byte;
} spi_if_t;

extern const spi_if_t SPI1_STM32;
```

**Fragmento 6. Instancia concreta exportada por spi\_if.c**

```
const spi_if_t SPI1_STM32 = {
    .gpio_init = SPI1_gpio_init,
    .init      = SPI1_init,
    .send_byte = SPI1_send_byte
};
```

### 7.1 ¿Qué gana el proyecto con esta interfaz?

Principalmente claridad y desacoplamiento. La aplicación no necesita recordar el nombre exacto de cada función del driver ni el archivo donde vive. Además, este patrón permite imaginar una evolución futura donde `SPI1_STM32` pudiera sustituirse por otra implementación, por ejemplo otro SPI o un driver simulado para pruebas.

## 8. Uso real del SPI dentro del proyecto

El valor del módulo se vuelve evidente cuando se observa su uso en los archivos de aplicación. El arranque del sistema inicializa primero los pines del SPI y luego el periférico. Más tarde, la librería del display emplea esa interfaz para transmitir tanto comandos de configuración como bytes de color o coordenadas.

**Fragmento 7. Arranque del bloque de visualización en main.c**

```
void ini_pantalla(void)
{
    GPIO_STM32.tft.init();
    PWM_STM32.tft_bl.init(10000, 450);
    SPI1_STM32.gpio_init();
    SPI1_STM32.init();
    GPIO_STM32.tft.cs_high();
    reset_TFT();
    ILI9341_init();
}
```

**Fragmento 8. Cooperación entre GPIO y SPI en ILI9341\_driver.c**

```
void ili_send_CMD(uint8_t cmd)
{
    GPIO_STM32.tft.dc_command();
    SPI1_STM32.send_byte(cmd);
}

void ili_send_data(uint8_t data)
{
    GPIO_STM32.tft.dc_data();
    SPI1_STM32.send_byte(data);
}
```

| Momento                    | Interfaz utilizada                  | Objetivo                                                                    |
|----------------------------|-------------------------------------|-----------------------------------------------------------------------------|
| Inicialización del sistema | <code>SPI1_STM32.gpio_init()</code> | Poner los pines PA5 y PA7 en su función alternativa AF5.                    |
| Inicialización del sistema | <code>SPI1_STM32.init()</code>      | Configurar CR1 y habilitar el periférico SPI1.                              |
| Envío de comandos TFT      | <code>SPI1_STM32.send_byte()</code> | Transmitir opcodes como 0x28, 0x11 o 0x2C.                                  |
| Envío de datos TFT         | <code>SPI1_STM32.send_byte()</code> | Transmitir parámetros, coordenadas y colores RGB565 al controlador gráfico. |

## 9. Observaciones didácticas y mejoras posibles

- La división entre `SPI1_gpio_init()` y `SPI1_init()` es correcta pedagógicamente, porque separa la configuración del puerto de la configuración del periférico.
- El comentario del prescaler no coincide con el valor cargado en CR1. Este tipo de desajuste es ideal para enseñar lectura crítica de registros.
- El proyecto resuelve un bus SPI de escritura principalmente bloqueante. Para un sistema con mayor carga gráfica, más adelante podría estudiarse el uso de DMA.
- Como el display no se consulta por lectura, una versión más avanzada del driver podría estudiar una estrategia explícita para la ruta RX si se desea evitar estados residuales de recepción.
- La interfaz `spi_if` es pequeña pero muy útil: introduce al estudiante a un patrón de abstracción frecuente en firmware modular.

## 10. Conclusión del módulo

El bloque SPI es el puente entre el procesamiento interno del STM32 y la visualización externa del proyecto. Gracias a él, la información estimada por el algoritmo RLS puede convertirse en comandos y datos gráficos para la pantalla TFT. Desde el punto de vista formativo, este módulo permite enseñar tres lecciones clave: cómo preparar pines en función alternativa, cómo configurar el registro CR1 de un SPI maestro y cómo envolver un driver bare-metal con una interfaz más limpia para el resto del firmware.